



PROJETO E ANÁLISE DE ALGORITMOS

AULA 3 – ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

3.1 ORDENAÇÃO POR INSERÇÃO

- A ordenação por inserção, algoritmo eficiente para ordenar um número pequeno de elementos. A ordenação por inserção funciona da mesma forma como muitas pessoas ordenam as cartas em um jogo de baralho. Começamos com a mão esquerda vazia e as cartas viradas para baixo, na mesa.
- Em seguida, retiramos uma carta de cada vez da mesa e a seguramos com a mão esquerda.
- Depois, com a mão direita, removemos uma carta de cada vez da mesa e a inserimos na posição correta na mão esquerda.

3.1 ORDENAÇÃO POR INSERÇÃO

- Você encontra a posição correta para uma carta comparando-a com cada uma das cartas que já estão na mão esquerda, começando da direita e movendo-se para a esquerda.
- Assim que for achada uma carta na mão esquerda cujo valor é menor ou igual à carta que você está segurando na sua mão direita, insira essa carta imediatamente à direita da carta em sua mão esquerda.



3.1 ORDENAÇÃO POR INSERÇÃO

- Se todas as cartas em sua mão esquerda tiverem valores maiores que a carta na sua mão direita, então coloque essa carta na posição mais à esquerda de sua mão esquerda. Em todas as vezes, as cartas que seguramos na mão esquerda estão ordenadas, e essas cartas eram originalmente as que estavam na parte superior da pilha sobre a mesa.
- O pseudocódigo dessa ordenação utiliza dois parâmetros: um vetor A contendo os valores a serem ordenados e o número n de valores de ordenação. Os valores ocupam as posições de $A[1]$ até $A[n]$ do vetor, que indicamos como $A[1:n]$. Quando o procedimento termina, o vetor $A[1..n]$ contém os valores originais, mas ordenados.

3.1 ORDENAÇÃO POR INSERÇÃO

ORDENA-POR-INSERÇÃO(A , n)

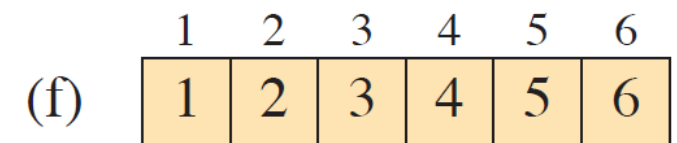
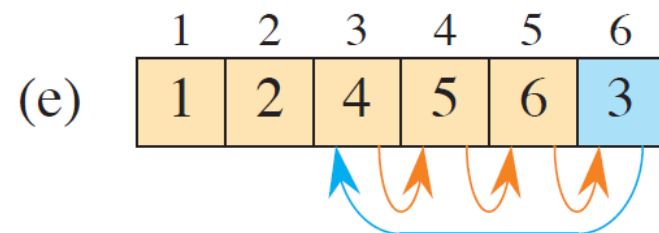
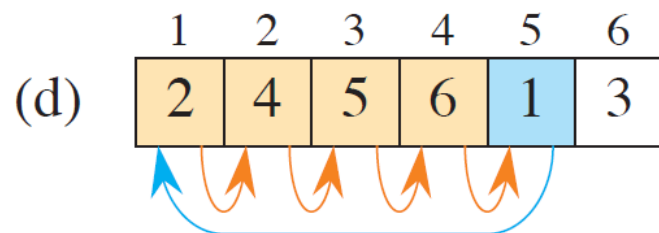
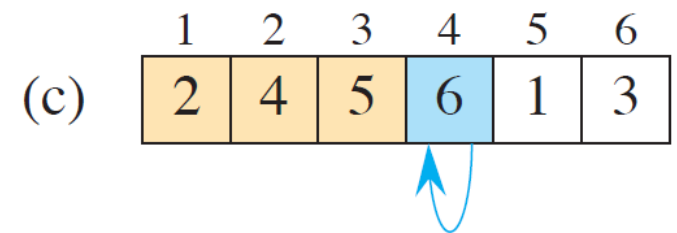
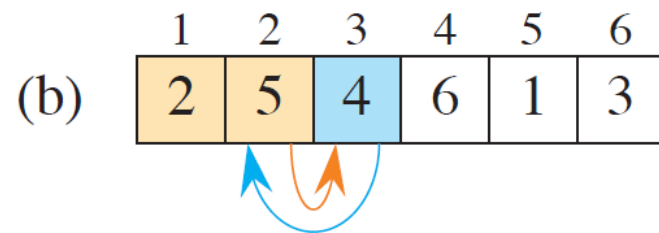
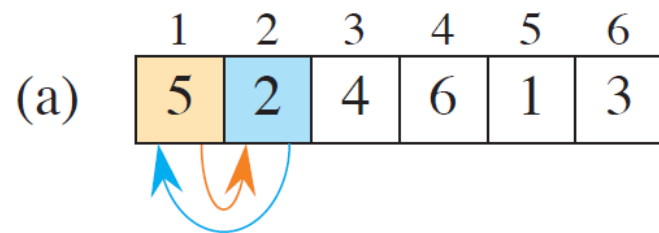
1. para $i = 2$ até n
2. chave = $A[i]$
3. //Inserir $A[i]$ no subvetor ordenado $A[1:i - 1]$.
4. $j = i - 1$
5. enquanto $j > 0$ e $A[j] > \text{chave}$
6. $A[j + 1] = A[j]$
7. $j = j - 1$
8. $A[j + 1] = \text{chave}$

3.1 ORDENAÇÃO POR INSERÇÃO

- O índice i indica a “carta atual” que está sendo inserida na mão. No início de cada iteração do laço for, indexado por i , o subvetor (uma parte contígua do vetor) que consiste nos elementos $A[1 : i - 1]$ (que é $A[1]$ até $A[i - 1]$) constitui a mão ordenada atualmente, e o subvetor remanescente $A[i + 1 : n]$ (elementos $A[i + 1]$ até $A[n]$) corresponde à pilha de cartas que ainda está sobre a mesa.
- Na verdade, os elementos $A[1 : i - 1]$ são os que estavam originalmente nas posições 1 a $i - 1$, mas agora em sequência ordenada.

3.1 ORDENAÇÃO POR INSERÇÃO

- Iterações do laço for das linhas 1–8



3.1 ORDENAÇÃO POR INSERÇÃO

- Como podemos provar que o algoritmo funciona?
- O que devemos demonstrar?
- Como devemos enunciar o que acontece no algoritmo?
- Qual a base matemática que servirá de apoio?

3.2 INVARIANTE DE LAÇO

- O funcionamento do algoritmo de ordenação por inserção ou qualquer outro algoritmo iterativo pode ser descrito por um invariante de laço.
- Para esse algoritmo, um invariante correto seria:

“No início de cada iteração para o laço for das linhas 1–8, o subvetor $A[1:i - 1]$ consiste nos elementos que estavam originalmente em $A[1:j - 1]$, porém em sequência ordenada.”

3.2 INVARIANTE DE LAÇO

- Usamos invariantes de laço para nos ajudarem a entender por que um algoritmo é correto. Devemos mostrar três detalhes sobre um invariante de laço:
- **Inicialização:** ele é verdadeiro antes da primeira iteração do laço.
- **Manutenção:** se ele for verdadeiro antes de uma iteração do laço, permanecerá verdadeiro antes da próxima iteração.
- **Término:** o laço termina e, quando ele termina, o invariante — normalmente, junto com o motivo para o laço ter terminado — nos fornece uma propriedade útil que ajuda a mostrar que o algoritmo está correto.

3.2 INVARIANTE DE LAÇO

- Quando as duas primeiras propriedades são válidas, o invariante de laço é verdadeiro antes de cada iteração do laço. (É claro que temos a liberdade de usar fatos confirmados além do invariante de laço em si para provar que ele permanece verdadeiro antes de cada iteração.)
- Uma prova do invariante de laço é uma forma de **indução matemática**, em que, para provarmos que uma propriedade é válida, provamos uma base e um passo de indução. Aqui, mostrar que o invariante é válido antes da primeira iteração equivale à base, e mostrar que o invariante é válido de uma iteração para outra corresponde ao passo de indução.

3.2 INVARIANTE DE LAÇO

- A terceira propriedade talvez seja a mais importante, visto que estamos usando o invariante de laço para mostrar a corretude.
- Normalmente, usamos o invariante de laço juntamente com a condição que provocou o término do laço.
- A indução matemática normalmente aplica a etapa indutiva indefinidamente, mas, em um invariante de laço, paramos a “indução” quando o laço termina.

3.2 INVARIANTE DE LAÇO

- Na ordenação por inserção:
 - **Inicialização:** começamos mostrando que o invariante de laço é válido antes da primeira iteração do laço, quando $i = 2$. Então, o subvetor $A[1 : i - 1]$ consiste apenas no único elemento $A[1]$, que é de fato o elemento original em $A[1]$. Além disso, esse subvetor está ordenado (trivialmente, é claro), e isso mostra que o invariante de laço é válido antes da primeira iteração do laço.

3.2 INVARIANTE DE LAÇO

- Na ordenação por inserção:
 - **Manutenção:** informalmente, o corpo do laço for funciona deslocando os valores em $A[i - 1]$, $A[i - 2]$, $A[i - 3]$, e assim por diante, uma posição para a direita até encontrar a posição adequada para $A[i]$ (linhas 4–7); nesse ponto ele insere o valor de $A[i]$ (linha 8). Então, o subvetor $A[1 : i]$ consiste nos elementos presentes originalmente em $A[1 : i]$, mas em sequência ordenada. Portanto, incrementar i (aumentar seu valor em 1) para a próxima iteração do laço for preserva o invariante de laço.
 - *Deveríamos ser mais formais e mostrar um invariante para o while das linhas 5-7, mas vamos nos concentrar apenas no laço externo por hora.

3.2 INVARIANTE DE LAÇO

- Na ordenação por inserção:
 - **Término:** por fim, examinamos o que acontece quando o laço termina. A variável de laço i começa em 2 e aumenta em 1 a cada iteração. Quando o valor de i ultrapassa n na linha 1, o laço termina. Isto é, o laço termina quando i é igual a $n + 1$. Substituindo i por $n + 1$ no enunciado do invariante de laço, temos que o subvetor $A[1 : n]$ consiste nos elementos originalmente contidos em $A[1 : n]$, mas em sequência ordenada. Portanto, o algoritmo está correto.

3.3 CONVENÇÕES DE PSEUDOCÓDIGO

- Recuos indicam estrutura de bloco
- As interpretações das construções de laço while, for e repeat-until e das construções condicionais if-else são semelhantes às das linguagens C, C++, Java, Python
- O símbolo “//” indica que o restante da linha é um comentário.
- Variáveis (como i, j e chave) são locais para o procedimento dado. Não usaremos variáveis globais sem indicação explícita.

3.3 CONVENÇÕES DE PSEUDOCÓDIGO

- Elementos de vetores são acessados especificando-se o nome do vetor seguido pelo índice entre colchetes. Por exemplo, $A[i]$ indica o i -ésimo elemento do vetor A .
- O primeiro índice válido dos vetores começa em 1 (pois as pessoas geralmente começam a contar em 1, não em 0).
- Elementos de vetores são acessados especificando-se o nome do vetor seguido pelo índice entre colchetes. Por exemplo, $A[i]$ indica o i -ésimo elemento do vetor A .

EXERCÍCIOS

- Considere o procedimento SOMA-VETOR a seguir. Ele calcula a soma de n números no vetor $A[1 : n]$. Indique um invariante do laço para esse procedimento e utilize suas propriedades de inicialização, manutenção e término para mostrar que o procedimento SOMA-VETOR retorna a soma dos números em $A[1 : n]$.

```
SOMA-VETOR(A, n)
1. soma = 0
2. para i = 1 até n
3.     Soma = soma + A[i]
4. return soma
```

EXERCÍCIOS

```
SOMA-VETOR(A, n)
1. soma = 0
2. para i = 1 até n
3.     Soma = soma + A[i]
4. return soma
```

- Invariante de laço: No início de cada iteração i do laço para, $\text{soma} = A[1] + A[2] + \dots + A[i-1]$.

EXERCÍCIOS

```
SOMA-VETOR(A, n)
1. soma = 0
2. para i = 1 até n
3.     Soma = soma + A[i]
4. return soma
```

- Invariante de laço: No início de cada iteração i do laço **para**, $\text{soma} = A[1] + A[2] + \dots + A[i-1]$.
- Agora, usaremos as propriedades de inicialização, manutenção e terminação da invariante do loop para mostrar que SOMA-VETOR retorna a soma dos números em $A[1:n]$.

EXERCÍCIOS

```
SOMA-VETOR(A, n)
1. soma = 0
2. para i = 1 até n
3.     Soma = soma + A[i]
4. return soma
```

- Inicialização: Ao entrar na primeira iteração, $i=1$. A soma $A[1]+A[2]+\dots+A[i-1]$ está vazia, pois $i-1=0$. A soma de nenhum termo é a identidade 0 para a adição.

EXERCÍCIOS

```
SOMA-VETOR(A, n)
1. soma = 0
2. para i = 1 até n
3.     Soma = soma + A[i]
4. return soma
```

- Manutenção: Suponha que a invariante do loop seja verdadeira, de forma que, ao entrar em uma iteração para um valor de i , $soma = A[1] + A[2] + \dots + A[i-1]$. A iteração adiciona $A[i]$ a soma e, em seguida, incrementa i , de forma que a invariante do loop se mantenha ao entrar na próxima iteração.
- Término: O loop termina quando $i=n+1$. De acordo com a invariante de laço, $soma=A[1]+A[2]+\dots+A[n]$. Portanto, soma, que o procedimento retorna, é de fato a soma de $A[1:n]$.

EXERCÍCIOS

- Considere o problema de busca:
- Entrada: uma sequência de n números $A = \langle a_1, a_2, \dots, a_n \rangle$ armazenada no vetor $A[1 : n]$ e um valor x .
- Saída: um índice i tal que x é igual a $A[i]$ ou o valor especial NULO, se x não aparecer em A .
- Escreva o pseudocódigo para busca linear, que faça a varredura do vetor, procurando por x . Usando um invariante de laço, prove que seu algoritmo está correto. Certifique-se de que seu invariante de laço satisfaz as três propriedades necessárias.

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

- Quanto tempo demora o procedimento ORDENA-POR-INSERÇÃO?
- Uma maneira de saber seria executá-lo em seu computador e **cronometrar** quanto tempo leva para ser executado. Claro, primeiro você teria que implementá-lo em uma **linguagem de programação** real, já que não é possível executar nosso pseudocódigo diretamente.

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

- O que esse teste de tempo lhe diria?
- Você descobriria quanto tempo a ordenação por inserção leva para ser executada em seu **computador específico**, com essa entrada específica, sob a implementação específica que você criou, com o compilador ou interpretador específico que você executou, com as bibliotecas específicas que você vinculou e com as tarefas em segundo plano específicas que estavam sendo executadas em seu computador simultaneamente com seu teste de tempo (como a verificação de informações recebidas em uma rede).

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

- Se você executar a ordenação por inserção novamente em seu computador com a mesma entrada, poderá até obter um tempo resultante diferente. Ao executar somente uma implementação da ordenação por inserção em apenas um computador e em apenas uma entrada, o que você seria capaz de determinar sobre o tempo de execução da ordenação por inserção se você lhe fornecesse uma entrada diferente, se a executasse em um computador diferente, ou se fosse implementá-la em uma linguagem de programação diferente?
- Não muito.

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

- Precisamos de **uma maneira de prever**, dada uma nova entrada, quanto tempo a ordenação por inserção levará.
- Em vez de cronometrarmos uma execução (ou mesmo várias execuções) da ordenação por inserção, podemos determinar quanto **tempo leva analisando o próprio algoritmo**.
- Examinaremos quantas vezes ele executa cada linha de pseudocódigo e quanto tempo cada linha de pseudocódigo leva para ser executada.

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

- Primeiro, vamos apresentar uma **fórmula precisa**, mas complicada, para obter o tempo de execução.
- Depois, vamos destilar a parte importante da fórmula usando uma notação conveniente, que pode nos ajudar a **comparar os tempos de execução de diferentes algoritmos** para o mesmo problema.
- Como analisamos a ordenação por inserção?
- Primeiro, vamos reconhecer que o tempo de execução depende da entrada.

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

- A ordenação por inserção pode demorar **quantidades de tempo diferentes** para ordenar duas sequências de entrada do mesmo tamanho, dependendo de até que ponto elas já estejam ordenadas.
- Embora o tempo de execução possa depender de muitas características da entrada, vamos nos concentrar em uma que possui o maior efeito, a saber, o tamanho da entrada, e descrever o tempo de execução de um programa em função do tamanho de sua entrada.

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

- Para fazermos isso, precisamos definir os termos “tempo de execução” e “tamanho da entrada” com mais cuidado.
- Também precisamos esclarecer se estamos discutindo o tempo de execução para uma entrada que deduz o comportamento no pior caso, o comportamento no melhor caso ou algum outro caso.
- A melhor noção de tamanho da entrada depende do problema que está sendo estudado. No caso de muitos problemas, como a ordenação a medida mais natural é o número de itens na entrada — por exemplo, a quantidade n dos itens sendo ordenados.

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

- Para muitos outros problemas, como a multiplicação de dois inteiros, a melhor medida do tamanho da entrada é o número total de bits necessários para representar a entrada em notação binária comum.
- Por vezes, é mais apropriado descrever o tamanho da entrada com mais do que apenas um número. Por exemplo, se a entrada para um algoritmo é um grafo, normalmente caracterizamos o tamanho da entrada pelos números de vértices e de arestas no grafo.

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

- O tempo de execução de um algoritmo com determinada entrada é o número de instruções e acessos a dados executados.
- O modo como podemos considerar esses custos deverá ser independente de qualquer computador específico, mas estar dentro da estrutura do modelo de RAM.
- Uma quantidade de tempo constante é exigida para executar cada linha do nosso pseudocódigo. Uma linha pode demorar uma quantidade de tempo diferente de outra linha, mas consideraremos que cada execução da k -ésima linha leva um tempo c_k , em que c_k é uma constante.

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

- Para analisarmos o procedimento ORDENA-POR-INSERÇÃO, vamos apresentá-lo com o custo de tempo de cada instrução e o número de vezes que cada instrução é executada.
- Para cada $i = 2, 3, \dots, n$, seja t_i o número de vezes que o teste do laço enquanto na linha 5 é executado para aquele valor de i . Quando um laço para ou enquanto termina da maneira usual — porque o teste no cabeçalho do laço resultou em FALSO —, o teste é executado uma vez a mais do que o corpo do laço.

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

- O tempo de execução do algoritmo é a soma dos tempos de execução para cada instrução executada.
- Uma instrução que demanda c_k passos para ser executada e é executada m vezes contribuirá com $c_k m$ para o tempo de execução total.
- Normalmente, indicamos o tempo de execução de um algoritmo sobre uma entrada de tamanho n com $T(n)$. Para calcularmos $T(n)$, o tempo de execução de ORDENA-POR-INSERÇÃO de uma entrada de n valores, somamos os produtos das colunas custo e vezes.

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

ORDENA-POR-INSERÇÃO(A , n)

1. para $i = 2$ até n
2. chave = $A[i]$
3. //Inserir $A[i]$ no subvetor ordenado $A[1:i-1]$.
4. $j = i - 1$
5. enquanto $j > 0$ e $A[j] > \text{chave}$
6. $A[j + 1] = A[j]$
7. $j = j - 1$
8. $A[j + 1] = \text{chave}$

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

ORDENA-POR-INSERÇÃO(A, n)

1. para $i = 2$ até n
2. chave = $A[i]$
3. //Inserir $A[i]$ no subvetor ordenado $A[1:i-1]$.
4. $j = i - 1$
5. enquanto $j > 0$ e $A[j] > \text{chave}$
6. $A[j + 1] = A[j]$
7. $j = j - 1$
8. $A[j + 1] = \text{chave}$

custo

vezes

c_1

c_2

0

c_4

c_5

c_6

c_7

c_8

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

ORDENA-POR-INSERÇÃO(A, n)

1. para $i = 2$ até n
2. chave = A[i]
3. //Inserir A[i] no subvetor ordenado A[1:i-1].
4. $j = i - 1$
5. enquanto $j > 0$ e $A[j] > \text{chave}$
6. $A[j + 1] = A[j]$
7. $j = j - 1$
8. $A[j + 1] = \text{chave}$

custo	vezes
C_1	n
C_2	$n-1$
0	$n-1$
C_4	$n-1$
C_5	$\sum_{i=2}^n t_i$
C_6	$\sum_{i=2}^n (t_i - 1)$
C_7	$\sum_{i=2}^n (t_i - 1)$
C_8	$n-1$

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

	custo	vezes
$T(n) = c_1 n + c_2(n - 1) + c_4(n - 1) +$	c_1	n
	c_2	$n-1$
	0	$n-1$
$c_5 \sum_{i=2}^n t_i + c_6 \sum_{i=2}^n (t_i - 1) + c_7 \sum_{i=2}^n (t_i - 1)$	c_4	$n-1$
	c_5	$\sum_{i=2}^n t_i$
	c_6	$\sum_{i=2}^n (t_i - 1)$
$+ c_8(n - 1)$	c_7	$\sum_{i=2}^n (t_i - 1)$
	c_8	$n-1$

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

- Mesmo para entradas de determinado tamanho, o tempo de execução de um algoritmo pode depender de qual entrada desse tamanho é dada.
- Por exemplo, em ORDENA-POR-INSERÇÃO, o melhor caso ocorre se o vetor já está ordenado. Nesse caso, toda vez que a linha 5 é executada, o valor de chave — o valor originalmente em $A[i]$ — já é maior ou igual a todos os valores em $A[1 : i - 1]$, de modo que o laço enquanto das linhas 5–7 sempre termina no primeiro teste na linha 5.

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

- Portanto, temos que $t_i = 1$ para $i = 2, 3, \dots, n$, e o tempo de execução do melhor caso é dado por:
- $T(n) = c_1n + c_2(n - 1) + c_4(n - 1) + c_5(n - 1) + c_8(n - 1)$
- $T(n) = (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8)$
- Podemos expressar esse tempo de execução como $an + b$ para constantes a e b que dependem dos custos de instrução c_k (em que $a = c_1 + c_2 + c_4 + c_5 + c_8$ e $b = -(c_2 + c_4 + c_5 + c_8)$). Assim, o tempo de execução é uma **função linear** de n .
- $T(n) = an + b$

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

- O pior caso resulta quando o vetor está disposto em ordem inversa — ou seja, em ordem decrescente.
- O procedimento deve comparar cada elemento $A[i]$ com cada elemento do subvetor ordenado inteiro, $A[1 : i - 1]$, e então, $t_i = i$ para $i = 2, 3, \dots, n$. (O procedimento descobre que $A[j] > \text{chave}$ toda vez que a linha 5 é executada, e o laço enquanto termina somente quando j alcança o valor 0.)

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

- Observando que $\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$ $\sum_{j=2}^n (j-1) = \frac{n(n-1)}{2}$

descobrimos que, no pior caso, o tempo de execução de ORDENA-POR-INSERÇÃO é

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) \\ &+ c_6 \left(\frac{n(n-1)}{2} - 1 \right) + c_7 \left(\frac{n(n-1)}{2} - 1 \right) + c_8(n-1) \end{aligned}$$

3.4 ANÁLISE DA ORDENAÇÃO POR INSERÇÃO

- $$T(n) = \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right)n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right)n - (c_2 + c_4 + c_5 + c_8)$$
- Podemos expressar esse tempo de execução do pior caso como $an^2 + bn + c$ para constantes a , b e c que, mais uma vez, dependem dos custos de instrução c_k (agora, $a = c_5/2 + c_6/2 + c_7/2$, $b = c_1 + c_2 + c_4 + c_5/2 - c_6/2 - c_7/2 + c_8$, e $c = -(c_2 + c_4 + c_5 + c_8)$). Portanto, o tempo de execução é uma **função quadrática** de n .
- $T(n) = an^2 + bn + c$

3.5 ANÁLISE DO PIOR CASO E DO CASO MÉDIO

- Em geral, nos preocupamos (ou damos maior importância) apenas para o pior caso:
 - O tempo de execução do pior caso de um algoritmo estabelece **um limite superior** no tempo de execução para qualquer entrada. Conhecê-lo nos dá uma garantia de que o algoritmo nunca demorará mais do que esse tempo. Não precisamos fazer nenhuma suposição sobre o tempo de execução esperando que ele nunca seja muito pior. Esse recurso é especialmente importante para a computação em tempo real, em que as operações precisam ser concluídas dentro de um prazo.

3.5 ANÁLISE DO PIOR CASO E DO CASO MÉDIO

- Em geral, nos preocupamos (ou damos maior importância) apenas para o pior caso:
 - Para alguns algoritmos, o pior caso ocorre com bastante frequência. Por exemplo, na pesquisa de um banco de dados em busca de determinada informação, o pior caso do algoritmo de busca frequentemente ocorre quando a informação não está presente no banco de dados. Em algumas aplicações, a busca de informações ausentes pode ser frequente.

3.5 ANÁLISE DO PIOR CASO E DO CASO MÉDIO

- Em geral, nos preocupamos (ou damos maior importância) apenas para o pior caso:
- Muitas vezes, o **“caso médio”** é quase tão ruim quanto o pior caso. Suponha que escolhemos n números aleatoriamente e aplicamos a ordenação por inserção. Quanto tempo transcorrerá até que o algoritmo determine o lugar no subvetor $A[1 : i - 1]$ em que deve ser inserido o elemento $A[i]$? Em média, metade dos elementos em $A[1 : i - 1]$ é menor que $A[i]$ e metade dos elementos é maior. Portanto, em média, $A[i]$ é comparado com apenas metade do subvetor $A[1 : i - 1]$ e, portanto, t_i é aproximadamente $i/2$. Resulta que o tempo de execução obtido para o caso médio é uma função quadrática do tamanho da entrada, exatamente o que ocorre com o tempo de execução do pior caso.

3.6 ORDEM DE CRESCIMENTO

- Usamos algumas abstrações simplificadoras para facilitar nossa análise do procedimento ORDENA-POR-INSERÇÃO.
- Primeiro, ignoramos o custo real de cada instrução, usando as constantes c_k para representar esses custos.
- Ainda assim, os tempos de execução no pior e no melhor caso, nas equações anteriores, são um tanto difíceis de se manejar. As constantes nessas expressões nos dão **mais detalhes do que realmente necessitamos**.

3.6 ORDEM DE CRESCIMENTO

- É por isso que expressamos o tempo de execução do melhor caso como $an + b$, com as constantes a e b que dependem dos custos de instrução c_k , e expressamos o tempo de execução do pior caso como $an^2 + bn + c$, com constantes a , b e c que dependem também dos custos de instrução.
- Desse modo, ignoramos não apenas os custos reais de instrução, mas também os custos abstratos c_k .

3.6 ORDEM DE CRESCIMENTO

- Podemos simplificar ainda mais com a ideia de taxa de crescimento, ou ordem de crescimento, do tempo de execução que realmente nos interessa.
- Consideramos apenas o termo inicial de uma fórmula (por exemplo, an^2), já que os termos de ordem mais baixa são relativamente insignificantes para grandes valores de n .
- Também ignoramos o coeficiente constante do termo inicial, visto que fatores constantes são menos significativos que a taxa de crescimento na determinação da eficiência computacional para grandes entradas.

3.6 ORDEM DE CRESCIMENTO

- No caso do tempo de execução da ordenação por inserção, quando ignoramos os termos de ordem mais baixa e o coeficiente constante do termo inicial, resta apenas o fator de n^2 do termo inicial.
- Esse fator, n^2 , é de longe a parte mais importante do tempo de execução.
- Por exemplo, suponha que um algoritmo implementado em uma máquina específica leve $n^2/100 + 100n + 17$ microssegundos com uma entrada de tamanho n .

3.6 ORDEM DE CRESCIMENTO

- Embora os coeficientes de $1/100$ para o termo n^2 e 100 para o termo n difiram em quatro ordens de grandeza, o termo $n^2/100$ domina o termo $100n$ quando n ultrapassa 10.000 .
- Embora 10.000 possa parecer grande, é um número menor do que a população de uma cidade média.

3.6 ORDEM DE CRESCIMENTO

- Para destacar a ordem de crescimento do tempo de execução, temos uma notação especial que usa a letra grega Θ (teta).
- A ordenação por inserção tem um tempo de execução do pior caso igual a $\Theta(n^2)$ (lido como “teta de n ao quadrado” ou apenas “teta n ao quadrado”).
- Também escrevemos que a ordenação por inserção tem um tempo de execução do melhor caso igual a $\Theta(n)$ (“teta de n” ou “teta n”).

3.6 ORDEM DE CRESCIMENTO

- Em geral, consideramos que um algoritmo é mais eficiente que outro se seu tempo de execução do pior caso apresenta uma ordem de crescimento mais baixa.
- Em razão de fatores constantes e termos de ordem mais baixa, um algoritmo cujo tempo de execução tenha uma ordem de crescimento mais alta pode demorar menos tempo para pequenas entradas do que um algoritmo cuja ordem de crescimento seja mais baixa.

3.6 ORDEM DE CRESCIMENTO

- Porém, para entradas suficientemente grandes, um algoritmo cujo tempo de execução no pior caso é $\Theta(n^2)$, por exemplo, leva menos tempo no pior caso que um algoritmo cujo tempo de execução no pior caso é $\Theta(n^3)$.
- Independentemente das constantes ocultas pela notação Θ , há sempre algum número, digamos, n_0 , tal que, para todos os tamanhos de entrada $n \geq n_0$, o algoritmo $\Theta(n^2)$ supera o algoritmo $\Theta(n^3)$ no pior caso.